Generative AI

Deepika Kanchi

October 2026

Contents

1 Introduction **2**
1.1 How It Works . . . 2
1.2 Examples of Generative AI . . . 2

2 Types of Generative AI Models **2**
2.1 Transformers or Autoregressive Models . . . 2
2.1.1 What Does "Auto regressive" Mean? . . . 2
2.1.2 Formula . . . 3
2.1.3 Step 1: Input Sequence . . . 3
2.1.4 Step 2: Probability of a Sequence . . . 3
2.1.5 Step 3: Token Embeddings . . . 3
2.1.6 Step 4: Add Positional Encoding . . . 4
2.1.7 Step 5: Self-Attention . . . 4
2.1.8 Step 6: Next Token Prediction . . . 4
2.1.9 Step 7: Select Next Token . . . 5
2.1.10 Example . . . 5
2.1.11 Use cases of this type of generative ai . . . 5
2.2 Diffusion Models . . . 5
2.2.1 Forward Diffusion Process . . . 6
2.2.2 Reverse Diffusion Process . . . 6
2.2.3 Applications . . . 6
2.2.4 Popular Diffusion Models . . . 7
2.3 VAEs and GANs . . . 7

3 Applications of Generative AI **7**

1 Introduction

Generative AI is a form of artificial intelligence that creates new content—such as text, images, audio, video, or code—by learning patterns from large datasets and generating original outputs based on those patterns. It uses models like transformers, GANs, and diffusion models to produce realistic and context-aware content.

1.1 How It Works

Generative AI typically involves:

- Training on huge volumes of raw data to learn patterns
- Tuning the model for specific tasks.
- Generation of new content based on user prompts.

1.2 Examples of Generative AI

- Text: ChatGPT, Claude, Gemini.
- Images: Midjourney v7, DALL·E, Stable Diffusion.
- Video: OpenAI Sora 2, Runway Gen-4.
- Audio/Voice: Suno, ElevenLabs.
- Code: Cursor, GitHub Copilot.

2 Types of Generative AI Models

2.1 Transformers or Autoregressive Models

An auto regressive model generates text one token (word/sub word) at a time. Each new token is predicted using all the tokens that came before it.

2.1.1 What Does "Auto regressive" Mean?

- **Auto** = self
- **Regressive** = uses previous outputs to predict the next output

An auto regressive model predicts the next token using its own previously generated tokens.

2.1.2 Formula

$P(w_1, w_2, w_3, \ldots, w_n) = P(w_1) \times P(w_2 \mid w_1) \times P(w_3 \mid w_1, w_2) \times \cdots \times P(w_n \mid w_1, w_2, \ldots, w_{n-1})$

- Predict the first word.
- Predict the second word given the first word.
- Predict the third word given the first two words.
- Continue this process until the entire sequence is generated.

sectionMathematics of Autoregressive Transformers An autoregressive transformer generates text by predicting the next token based on previously generated tokens.

2.1.3 Step 1: Input Sequence

Consider the input sequence:

$$X = (x_1, x_2, x_3, \ldots, x_n)$$

where x_i represents the ith token. For example:

$$(\text{I}, \text{love}, \text{machine})$$

The model predicts the next token after "machine".

2.1.4 Step 2: Probability of a Sequence

Using the chain rule of probability, the probability of a sequence is:

$$P(x_1, x_2, \ldots, x_n) = \prod_{t=1}^{n} P(x_t \mid x_1, \ldots, x_{t-1})$$

This means:

$$P(x_1) \times P(x_2|x_1) \times P(x_3|x_1, x_2) \times \cdots$$

Each token depends on all previous tokens.

2.1.5 Step 3: Token Embeddings

Each token is converted into a vector representation:

$$e_t = \text{Embedding}(x_t)$$

Example:

$$x_t = \text{"love"}$$
$$e_t = [0.2, 0.8, 0.5, \ldots]$$

2.1.6 Step 4: Add Positional Encoding

Since transformers do not understand order naturally, positional information is added:

$$h_t = e_t + p_t$$

where:

- e_t = token embedding
- p_t = positional encoding

2.1.7 Step 5: Self-Attention

The transformer creates:

$$Q = HW_Q$$
$$K = HW_K$$
$$V = HW_V$$

where:

- Q = Query matrix
- K = Key matrix
- V = Value matrix

Attention scores are computed as:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

The attention mechanism helps the model focus on important previous tokens.

2.1.8 Step 6: Next Token Prediction

The transformer outputs a hidden representation:

$$h$$

which is converted into logits:

$$z = Wh + b$$

The probability of each next token is obtained using Softmax:

$$P(y = i) = \frac{e^{z_i}}{\sum_{j=1}^{V} e^{z_j}}$$

where V is the vocabulary size.

2.1.9 Step 7: Select Next Token

The token with the highest probability is chosen:

$$y = \arg\max_i P(y = i)$$

Example:

$$P(\text{programming}) = 0.85$$

$$P(\text{snake}) = 0.10$$

$$P(\text{city}) = 0.05$$

The model generates:

programming

2.1.10 Example

Consider the sentence:

"I love machine learning"

The model computes:

$$P(\text{I}, \text{love}, \text{machine}, \text{learning}) = P(\text{I}) \times P(\text{love}|\text{I}) \times P(\text{machine}|\text{I}, \text{love}) \times P(\text{learning}|\text{I}, \text{love}, \text{machine})$$

Thus, each new word is generated based on all previously generated words.

2.1.11 Use cases of this type of generative ai

Auto regressive transformers are widely used for text generation, chat bots, code generation, code completion, translation, summarization, content creation, question answering, and virtual assistants. They work by generating one token at a time based on previously generated tokens, making them highly effective for applications such as Chat GPT, GitHub Copilot, document summarization tools, and AI-powered customer support systems.

2.2 Diffusion Models

Diffusion Models are Generative AI models that create data by starting from random noise and gradually removing that noise through multiple denoising steps until a meaningful output is produced. During training, they learn to reverse a noise-adding (diffusion) process. Models such as Stable Diffusion and DALL·E use this approach to generate highly realistic images from text prompts. Besides image generation, diffusion models are also used for inpainting, image editing, style transfer, audio generation, and other content-creation tasks.

2.2.1 Forward Diffusion Process

In the forward process, Gaussian noise is added to the input data over multiple time steps. Given an image x_0, the noised sample at time step t is given by:

$$q(x_t|x_{t-1}) = \mathcal{N}\left(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I\right)$$

where:

- x_{t-1} is the data at the previous step,
- x_t is the noised data at step t,
- β_t is the noise schedule,
- $\mathcal{N}$ represents a Gaussian distribution.

As the number of diffusion steps increases, the data gradually becomes pure random noise.

2.2.2 Reverse Diffusion Process

The objective of the model is to learn the reverse process and reconstruct the original data from noise. The reverse process is defined as:

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}\left(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t)\right)$$

where:

- μ_θ is the predicted mean,
- Σ_θ is the predicted variance,
- θ represents the model parameters.

The model iteratively removes noise until a realistic sample is generated.

2.2.3 Applications

Diffusion Models are widely used in:

- Text-to-image generation
- Image super-resolution
- Image inpainting
- Medical image synthesis
- Video generation
- Scientific simulations

2.2.4 Popular Diffusion Models

Some well-known diffusion-based generative models include:

- Stable Diffusion
- DALL-E 2
- Imagen
- Midjourney
- Latent Diffusion Models (LDMs)

2.3 VAEs and GANs

Variational Autoencoders (VAEs) and Generative Adversarial Networks (GANs) are foundational Generative AI architectures used to generate new data samples such as images, audio, and text. A VAE consists of an encoder and decoder that learn a probabilistic latent representation of data, enabling smooth interpolation and controllable generation. In contrast, a GAN consists of two competing networks: a generator that creates synthetic data and a discriminator that attempts to distinguish real data from generated data. This adversarial training process allows GANs to produce highly realistic outputs. While VAEs provide stable training and well-structured latent spaces, GANs are capable of generating sharper and more lifelike images but may suffer from training instability and mode collapse.

3 Applications of Generative AI

- Text: Powers chatbots, virtual assistants, content creation, document summarization and even code generation tools like GitHub Copilot.
- Images: Used in digital art, product design, fashion, advertising and medical imaging to create visuals that are close to real-world examples.
- Audio and Speech: Enables natural-sounding voice assistants, multilingual dubbing, music composition, personalized voice cloning and accessibility tool s.
- Video: Supports animation, movie special effects, gaming, marketing videos and realistic training simulations.
- Business Use Cases: Enhances customer support with AI agents, boosts knowledge discovery using RAG systems, accelerates drug discovery, assists in financial forecasting and improves data-driven decision-making.